HacdsDeFi RPC Documentation

2026-11-11 Contract #3000 For developers

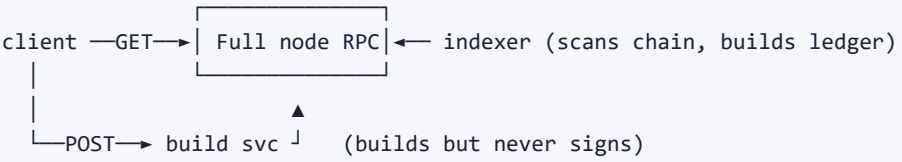
> This document describes the Hacash mainnet contract `HacdsDeFi3000.fitsh` (address `T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH`, asset serial 3000, decimal 0).
>
> **This document contains no #2000-era interfaces.** The oracle, scoring certificates, and admin functions **no longer exist** in the new contract — not because permissions were revoked, but because those functions were never written into it.

---

0. Services at a Glance

hacds involves four HTTP services. The first two read the chain; the other two are conveniences.

Service	Address	Protocol	Purpose
Full node RPC	`node.hacds.com`	HTTP/JSON	Chain data, sandbox read-only calls, verify/broadcast
API proxy	`api.hacds.com`	HTTP/JSON	Forwards to the node, adds CORS + HTTPS
Indexer	`indexer.hacds.com`	HTTP/JSON	Borrower ledger (who is liquidatable)
Build service	`build.hacds.com`	HTTP/JSON	Turns human input into an unsigned transaction



Why an API proxy: the node does not return CORS headers, so browsers are blocked from calling it directly. `api.hacds.com` is a Cloudflare Worker that only forwards and adds headers — semantics are unchanged. The origin ports are not 80/443, so Cloudflare reaches the origin through a Tunnel.

Response convention: every endpoint returns JSON with a top-level `ret`. `ret: 0` means success; non-zero means failure (usually with `err`).

> ⚠ **Failures also return HTTP 200.** You must check `ret` in the body, not the status code.

Why nodes are subdomains, not `IP:port`: Cloudflare Workers can only reach ports 80 and 443. Any `IP:8082` style address is rejected before a connection is attempted (`error code: 1003`). Everything external therefore goes through `*.hacds.com` subdomains backed by a Tunnel.

---

1. Full Node API

1.1 Query block height

```
GET /query/block/height/latest
```

```
{"ret":0,"height":798812}
```

VM status (enabled, current height):

```
GET /api/vm/status
```

```
{"ret":0,"vm":"enabled","height":798812}
```

1.2 Query average fee

Used to estimate transaction fees.

```
GET /query/fee/average
```

```
{"ret":0,"purity":"6024000000","purity_unit":232}
```

`purity` = `fee / billing_size (bytes)`. Node admission requires `purity ≥ min_purity`.

1.3 Query balances (HAC / diamonds / assets)

```
GET /query/balance?address={address}&assets=true&asset_meta=true&diamonds=true
```

All three flags are optional:

Flag	Effect
`assets=true`	Include the asset list (hacds lives here)
`asset_meta=true`	Include asset name / supply metadata

Flag	Effect
`diamonds=true`	Include the HACD diamond list

```
{
  "ret": 0,
  "list": [{
    "address": "1P2tsn...",
    "hacash": "1234567:248",
    "diamond": 3,
    "assets": [{"serial": 3000, "amount": "100000000"}]
  }]
}
```

Amounts use the `<value>:<unit>` form: `5:249` means 50 HAC

(5 × 10^(249−248)), not 5.0038. See §6.3 for the conversion.

1.4 Query diamond details

```
GET /query/diamond?name={6-char diamond name}
```

Returns the diamond's number, visual gene, owner, and more. Scoring needs

`number` and `visual_gene`, both found here.

1.5 Sandbox read-only call (the primary way to read contract state)

```
GET /query/contract/sandboxcall?contract={contract}&function={function}&params={params}&caller=
{caller}
```

Read-only execution: no state is written, no transaction is sent, no fee is paid.

Observed response:

```
{"ret":0,"use_gas":377,"gas_use":{"compute":132,"resource":209,"storage":36},"ret_val":120200677111}
```

Field	Meaning
`ret`	0 = call succeeded
`ret_val`	Function return value
`use_gas` / `gas_use`	Gas consumed, for estimation

> ⚠ **Before deployment this returns `NotFindContract`.** That is expected:
> the contract is deployed at height 800,000. It becomes callable afterwards.

1.5.1 Two pitfalls you must know

Pitfall 1: omitting `caller` fails.

```
{"ret":1,"err":"[revert] address \"1111...\" balance 0:0 is insufficient, at least 559555:242"}
```

The default caller has zero balance and cannot prepay gas. **Pass `caller={contract address}`** — a contract calling itself needs no balance.

Pitfall 2: sandbox concurrency per IP is 2. Excess requests are **rejected outright**, not queued:

```
{"ret":1,"err":"sandbox_per_ip_concurrency_exceeded"}
```

Clients must **serialize with backoff**. Do not fire a dozen with `Promise.all`.

1.5.2 Parameter format

`params` is `value:type` separated by commas. `:` must be URL-encoded as `%3A`.

Type	Example
`u128`	`100000000:u128`
`address`	`1P2tsn...:address`
`bytes`	`WTYUIA:bytes`

1.5.3 Complete read-only function list

AMM and liquidity

Function	Params	Returns	Notes
`amm_reserves`	—	u128	Pool HAC (**zhu**)
`amm_reserve_ds`	—	u128	Pool hacds (**count**)
`pool_price`	—	u128	Pool price = `amm:hac × 1e8 / amm:ds`
`lp_total`	—	u128	Total LP shares
`lp_share_of`	`who:address`	u128	An address's LP shares
`lp_pending_hac`	`who:address`	u128	Pending LP reward (HAC, zhu)
`lp_pending_ds`	`who:address`	u128	Pending LP reward (hacds, count)
`total_staked`	`kind:u128`	u128	kind `0` =HAC, `1` =hacds

Lending

Function	Params	Returns	Notes
`loan_debt_of`	`who:address`	u128	Principal (zhu), **excluding** unsettled interest
`loan_debt_now`	`who:address`	u128	Debt including interest (zhu)

Function	Params	Returns	Notes
`loan_coll_of`	`who:address`	u128	Locked collateral (hacds count)
`loan_health`	`who:address`	u128	**Health as basis points**, see below
`loan_status`	`who:address`	u128	Collateral ratio ×100, see below
`total_debt`	—	u128	Global debt (zhu)
`providers_total`	—	u128	Total lendable funds
`utilization`	—	u128	Current utilization (bps)
`borrow_coll_ratio`	—	u128	Borrow line percentage (`200`)
`liq_coll_ratio`	—	u128	Liquidation line percentage (`160`)
`borrow_paused`	—	u128	Non-zero = new borrowing paused
`ref_paused`	—	u128	Non-zero = price deviation circuit breaker active
`lendable_amm`	—	u128	Lendable portion of AMM (30%)
`net_claims`	—	u128	Total claims network-wide
`reserve_hac`	—	u128	Bad debt reserve
`deficit_hac`	—	u128	Deficit

How to read health correctly

`loan_health` returns	Meaning
`1000000`	No debt (sentinel the contract returns; treat as very healthy)
`> 10000`	Safe
`= 10000`	Exactly at the liquidation line
`< 10000`	**Liquidatable**
`< 9000`	Deep water; full liquidation allowed

Important: this value is computed with `coll_to_hac_liq()`, i.e. the **conservative price** `min(spot, smoothed reference)`, the same source as `loan_liquidate`. So you will never see "interface says safe, chain says liquidatable". Do not compute health yourself from `loan_status`; use this.

`loan_status` is the legacy view (`coll_hac × 100 / debt`, spot price), kept for compatibility:

Return	Meaning
`0`	No debt and no collateral
`999`	Collateral but no debt
other	Collateral ratio ×100

HACD pool

Function	Params	Returns	Notes
`hacd_score_total`	—	u128	Total HACD score in pool (already × `SCORE_SCALE`)
`hacd_count`	—	u128	Number of HACDs in pool
`hacd_avg_score`	—	u128	Average score (already × `SCORE_SCALE`)
`hacd_coverage`	—	u128	Coverage (bps, 10000 = 100%)
`quote_redeem`	`sc:u128`	tuple	Redemption quote, see below
`quote_score`	`name:bytes,vg:bytes,num:u128`	u128	Score by name + gene + number

`quote_redeem` returns: `tuple(charge, fee, premium_bps, ok)`

Element	Meaning
`charge`	Total deduction (principal + fee)
`fee`	The fee portion
`premium_bps`	Cherry-pick premium (bps)
`ok`	`1` = redeemable; `0` = the real transaction would return `61`

Price and factors

Function	Params	Returns	Notes
`ref_price_of`	—	u128	Smoothed reference price
`ref_price_zhu_per_ds`	—	u128	Reference price (zhu per hacds)
`ref_height`	—	u128	Block height of the last reference update
`hcut`	—	u128	HAC socialization factor (1e12 scale, only decreases)
`lcut`	—	u128	LP reward socialization factor

Accounts and treasury

Function	Params	Returns	Notes
`balance_hac`	`who:address`	u128	Effective HAC balance (zhu, already multiplied by hcut)

Function	Params	Returns	Notes
`balance_hac_nominal`	`who:address`	u128	Nominal share (zhu)
`balance_ds`	`who:address`	u128	Pending hacds quota (due)
`bad_of`	`who:address`	u128	Bad debt attributed to an address
`treasury_pending`	`kind:u128`	u128	Treasury pending sweep. `0` =HAC, `1` =hacds
`treasury_ds`	—	u128	Accumulated treasury hacds
`tot_hac`	—	u128	Total HAC (effective)
`tot_hac_nominal`	—	u128	Total HAC (nominal)
`lpr_hac_effective`	—	u128	Effective LP reward pool

Operations

Function	Params	Returns	Notes
`self_check`	—	u128	Internal consistency check, `0` = pass
`rent_key_group_count`	—	u128	Number of key groups (fixed `7`)
`rent_key_len_at`	`idx:u128`	u128	Key length of group idx
`rent_keys_group`	`klen:u128`	bytes	All keys of that length (feed directly to `rent_keys`)

1.5.4 Observed sample

```
GET {node}/query/contract/sandboxcall?contract=T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH
    &function=loan_health&params=1P2tsn...%3Aaddress&caller=T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH
→ {"ret":0,"use_gas":830,"gas_use":{"compute":363,"resource":323,"storage":144},"ret_val":1000000}
```

`ret_val: 1000000` = that address has no debt.

1.6 Query block summary

The indexer uses this to walk blocks looking for borrowers.

```
GET /query/block/intro?height={height}&tx_hash_list=true
```

1.7 Query transaction

```
GET /query/transaction?hash={tx hash}
```

1.8 Verify transaction (with human-readable description)

```
POST /util/transaction/check
Content-Type: application/json

{"tx": "040000... (hex)"}
```

On success it returns the parsed result and a readable action description:

```
{"ret":0,"describe":"[CallContract] T1A8... -> swap_hac_to_ds(...)"}
```

This endpoint does not sign or broadcast; it only verifies. Wallets use it as a final gate before broadcasting.

1.9 Broadcast transaction

```
POST /submit/transaction
Content-Type: application/json

{"tx": "040000... (hex)"}
```

```
{"ret":0}
```

> ⚠ `ret: 0` means **it reached the mempool only**, not that it is confirmed.
> Target block time is 300 seconds; poll `/query/transaction` on that cadence.

1.10 Contract storage cost parameters

```
GET /query/contract/storage_cost?contract={contract}
```

Used to estimate renewal cost.

---

2. Indexer API

The indexer scans the chain and maintains a ledger of "who has debt, who is liquidatable". **Wallets do not scan the chain themselves.**

2.1 Health check

```
GET /health
```

```
{"ok": true, "service": "hacds-indexer", "head": 798812, "watched": 0}
```

2.2 Ledger snapshot (the only endpoint most clients need)

```
GET /snapshot
```

Returns a health snapshot of every watched address:

```
{
  "head": 798812,
  "updated": 1762900000,
  "items": [
    {"addr": "1P2tsn...", "debt": "100000000", "coll": "300000000",
     "health": 16000, "ratio": 160, "liq": true}
  ]
}
```

Field	Meaning
`debt`	`loan_debt_now` (including interest)
`coll`	Collateral hacds count
`health`	Basis points, same scale as `loan_health`
`ratio`	Collateral ratio percentage
`liq`	`true` = currently liquidatable

2.3 Snapshot only (no sync)

```
GET /snapshot?cached=1
```

2.4 Force sync

```
POST /sync
```

Scans up to the latest block immediately. For operations; not normally needed.

2.5 Watch list maintenance

```
POST /watch/add?addr={address}
POST /watch/remove?addr={address}
```

The indexer discovers borrowers automatically; these fill gaps manually.

2.6 Issuance coverage monitoring

```
GET /backing
```

Compares "total due the contract records" against "hacds actually issued

on-chain", to detect anomalies.

2.7 Rent batches

```
GET /rent_batches
```

Returns key batches grouped by length that can be fed directly to `rent_keys`.

The keeper uses this.

2.8 Deployment

```
python3 hacds_indexer.py --node http://127.0.0.1:8082 \
    --contract T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH --port 8124
```

---

3. Build Service

This service translates "human-readable parameters" into "an unsigned transaction". **It holds no private key; the transaction it builds must be signed by the client.**

Source: `hacds_exchange/indexer/hacds_build_server.py`

3.0 Why it exists

Constructing a Hacash transaction requires precise fee / gas / action encoding, plus the wire format for diamond name lists. Having every client reimplement that guarantees the three clients will behave differently. Centralizing construction means the client only signs — **what gets signed changes, who signs does not.**

3.1 Health check

```
GET /health
```

```
{"ok": true, "service": "hacds-build", "version": "2026-11-11.2",
 "txbuild_ok": true, "contract": "T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH"}
```

`version` matters: Python does not hot-reload, so a changed script requires a process restart. If behavior and `version` disagree, check that the process was restarted.

3.2 Build a contract call

```
POST /api/build
Content-Type: application/json
```

```
{
  "mode": "call",
  "main": "1P2tsn...",
  "fn": "swap_hac_to_ds",
  "params": "100000000:u128,0:u128,0:u128",
  "fee": "0:248",
  "gasMax": 99
}
```

Field	Required	Notes
`mode`	No	`call` (default) or `transfer`
`main`	✅	Originating address (pays the fee)
`fn`	✅	Contract function name, whitelist in §3.4
`params`	No	`value:type` comma-separated, default `-`
`fee`	No	Form `0:248`
`gasMax`	No	Integer `1–99`, default `99`

Response:

```
{"ok": true, "unsigned_tx": "040000...", "fn": "swap_hac_to_ds",
 "fee": "0:248", "gas_max": 99}
```

3.3 Build a transfer

```
POST /api/build
{
  "mode": "transfer",
  "main": "1P2tsn...",
  "sub": "hacd-from-to",
  "fee": "0:248"
}
```

`sub` is a `hacds_txbuild transfer` subcommand; extra arguments follow the subcommand. `from` **is locked to the contract address** — a safety constraint preventing this service from constructing "transfer from an arbitrary address" transactions.

3.4 Function whitelist

```
swap_hac_to_ds · swap_ds_to_hac
add_liquidity · remove_liquidity · lp_harvest
loan_add_collateral · loan_borrow · loan_repay · loan_liquidate
rent_keys · topup_keys
```

Functions outside the whitelist are rejected. **It is a whitelist, not a blacklist** — new functions must be added explicitly.

3.5 Limits

Limit	Value	Why
Requests per IP per minute	30	Abuse prevention
Request body	8 KB	Params capped at 1024 bytes, at most 16 items
Diamond name list	At most 200	Matches `MAX_DEPOSIT_COUNT`
`gasMax`	1–99	Valid range for the contract and engine

3.6 Error codes

`error`	Meaning
`bad_json`	Body is not valid JSON
`bad_length`	Body empty or over 8 KB
`bad_main`	`main` is not a valid address
`bad_fee`	`fee` is not in `value:mantissa` form
`bad_gasMax`	`gasMax` is not an integer 1–99
`fn_not_allowed`	Function not in whitelist
`bad_params`	Parameter string malformed
`rate_limited`	Over 30 requests per minute
`build_timeout`	Engine timeout (30 seconds)
`engine_missing`	`hacds_txbuild` not found on the server

---

4. Exchange Listing Integration

4.0 One thing to remember: amount is a plain count

hacds has **decimal 0**. The `amount` returned on-chain is the count itself with **no conversion**.

```
{"serial": 3000, "amount": "100000000"}   →  100,000,000 hacds
```

The previous edition was decimal 8 and required dividing by 1e8. **Do not divide in this edition.**

4.1 Asset identity (for listing registration)

Field	Value
serial	`3000`
ticket	`HACDS`
name	`hacds`
decimal	`0`
supply	`5963298700000000`
issuer	`T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH`

⚠ There is also an asset with serial `1028` whose ticket is HACDS (name "HACD Fragments", someone else's). **Match on serial, not on ticket.**

4.2 Detecting deposits

Scan blocks for `TransferAssetFromTo` actions where `serial == 3000` and `to == your deposit address`.

```
GET /query/block/intro?height={H}&tx_hash_list=true
  → iterate tx_hash_list, GET /query/transaction?hash={h} for each
  → look at the asset transfer actions
```

Suggested confirmations: 3 blocks (about 15 minutes). Target block time is 300 seconds but varies.

4.3 Constructing withdrawals

A withdrawal is a standard asset transfer:

```
hacds_txbuild transfer {main} {ts} asset-from-to {from} {to} hacds {amount}
```

Or through the build service (`mode=transfer`, `sub=asset-from-to`).

4.4 Broadcast and confirmation

```
POST {node}/submit/transaction
{"tx": "...hex..."}
```

`ret:0` only means it reached the mempool. Poll `/query/transaction?hash=` by block height; it is confirmed once `block_height` appears.

4.5 Fees and gas

Item	Notes
fee format	`value:mantissa`, e.g. `0:248` (minimal), `2:247` (0.02 HAC)
fee formula	`fee = purity × billing_size`
`gasMax`	`1–99`, default 99. **Do not go below 99 for contract calls**

4.6 Three hard limits you must know

① **A transaction is at most 16,384 bytes.** This is a compile-time constant.

A 200-diamond deposit approaches the cap.

② **Sandbox concurrency per IP = 2.** Excess is rejected, not queued. Exchange

backends must serialize.

③ **Three gas buckets**: compute 18,009 / resource 6,100 / storage 111,911.

Exceeding one reports `OutOfGas(31): <bucket> N > M`. **Raising `gasMax` does not help** — that is the total budget; the per-bucket limit is a hard constraint.

> #3000 could raise "1 diamond per transaction" to "200 diamonds per transaction" precisely because scoring moved into an RV64IM kernel: 200 diamonds cost only 3,587 of `compute`, one bucket. The old FitSH version cost 3,270 of `resource` for a single diamond, so two exceeded 6,100.

4.7 Data needed for market making / quoting

```
GET {node}/query/contract/sandboxcall?contract={contract}&function=amm_reserves&caller={contract}
GET {node}/query/contract/sandboxcall?contract={contract}&function=amm_reserve_ds&caller={contract}
```

Spot price (zhu per hacds) = `amm_reserves × 1e8 / amm_reserve_ds`.

Fetching in bulk is easier:

Function	Returns at once
`ref_price_zhu_per_ds`	Smoothed reference price
`pool_price`	Pool price (×1e8)
`total_debt` / `utilization`	Lending utilization
`hacd_count` / `hacd_avg_score`	Pool size

4.8 Reconciliation

The on-chain identity:

```
sum of user due + AMM pool + treasury accumulation + LP reward pool
    == total hacds issued on-chain
```

Plus:

```
sum of effective HAC balances × hcut == sum of nominal shares
```

The indexer's `GET /backing` checks this on every sync. A mismatch means a bug.

4.9 Listing checklist

- [ ] Register with serial `3000`, decimal `0` (**not 8**)
- [ ] Deposits recognize serial 3000 only; ignore the same-ticket #1028
- [ ] Withdrawals use `asset-from-to` with `serial` either `hacds` or `3000`
- [ ] Confirmations ≥ 3
- [ ] Backend serializes sandbox calls (concurrency limit 2)
- [ ] Large withdrawals check whether the transaction hits the 16,384-byte cap
- [ ] Reconcile with `/backing` and the on-chain identity

---

5. Error Handling

5.1 Contract return codes

A non-zero `ret_val` from a sandbox call is a `return N` inside the contract.

General

Code	Meaning
`2`	Malformed parameter (e.g. HACD name list wire format)
`48`	Count out of range (0 diamonds / over 200 / redemption not exactly 1)
`5`	Score is 0
`53`	Would exceed total supply
`54`	Coverage insufficient (never triggers under 100% issuance)
`59`	Expired (`deadline`)
`61`	No HACD pool record / insufficient score (redemption)

AMM

Code	Meaning
`2`	One side is zero
`23`	Pool non-empty but shares round to zero (anti-rescale)
`24`	Computed shares are zero (ratio too small)

Code	Meaning
`25`	Insufficient LP shares
`26`	First deposit exceeds the `1e18` per-side cap
`56`	Slippage protection triggered (`min_out` / `min_share` not met)

Lending

Code	Meaning
`40`	Exceeds borrow limit (`(debt+hac) × 200% > coll_hac`)
`41`	Debt is zero (repay)
`50`	Global utilization over 50%
`51`	Borrowing paused (utilization full or price circuit breaker)
`52`	Exceeds single-borrower cap (20%)

Liquidation

Code	Meaning
`42`	That address has no debt
`43`	Not yet liquidatable
`44`	Below the minimum liquidation amount (1 HAC)
`45`	Collateral out below `min_coll_out`
`46`	Deep-water liquidation requires full repayment (`repay < deep_pay`)
`47`	Reference price uninitialized; liquidation refused

Admin functions do not exist

`set_fee` / `set_loan_rate` / `set_liquidate_bonus` / `set_paused` / `set_oracle`

are **not implemented** in #3000. Calling them returns:

```
CallNotExist(51)
```

That is not a permission failure — **the function does not exist.** Stop writing those calls.

5.2 Node / sandbox errors

Match by string; the common classes:

Raw message	Meaning	Handling
`NotFindContract`	No contract code at that address	Normal before deployment; check the address after
`invalid address argument` / `InvalidChecksum`	Bad address format or checksum	Re-fetch the full address
`balance ... is insufficient`	Caller cannot prepay gas	Pass `caller={contract address}`
`sandbox_per_ip_concurrency_exceeded`	Concurrency exceeded	Serialize + backoff
`resource gas limit exceeded`	Resource bucket exceeded	Split transactions; **raising gas_max does not help**
`compute gas limit exceeded`	Compute bucket exceeded	Split transactions
`storage gas limit exceeded`	Storage bucket exceeded	Fewer new keys or shorter rent
`OutOfGas(31)`	A bucket ran out	Read the bucket name in parentheses, split
`Arithmetic(90)`	Arithmetic under/overflow	Check inputs; this is a contract bug, please report
`CallNotExist`	Contract has no such function	Check the name (admin functions removed)
`state changed`	On-chain state changed mid-flight	Retry
`transaction parse failed`	Transaction body unparseable	Incomplete hex / bytes
`StorageNotActive`	Storage key expired	Call `rent_keys` to renew
`ECONNREFUSED` / `timeout`	Service unreachable	Check whether it is running

Rule of thumb: most of these are **HTTP 200 +** `{"ret":1,"err":"..."}`.

Checking only the status code will miss them.

---

6. Quick Reference

6.1 Endpoint summary

```
# Full node
GET  /query/block/height/latest
GET  /query/fee/average
GET  /query/balance?address={A}&assets=true&diamonds=true
GET  /query/diamond?name={N}
GET  /query/contract/sandboxcall?contract={C}&function={F}&params={P}&caller={C}
```

```
GET  /query/block/intro?height={H}&tx_hash_list=true
GET  /query/transaction?hash={H}
POST /util/transaction/check        {"tx": "hex"}
POST /submit/transaction            {"tx": "hex"}

# Indexer
GET  /health
GET  /snapshot
POST /sync
GET  /backing
GET  /rent_batches

# Build service
GET  /health
POST /api/build
```

6.2 Constants

Constant	Value
Contract address	`T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH`
Contract nonce	`2`
Treasury	`1FbwjELGMmaomNQq3ZZ6hdgQ17GRGwzAzK`
hacds serial	`3000`
hacds decimal	`0`
hacds total supply	`5963298700000000`
`SCORE_SCALE`	`100000000` (1e8)
`MAX_DEPOSIT_COUNT`	`200`
`FEE_BPS`	`50` (0.5%)
`BORROW_RATIO_BPS`	`20000` (200%)
`LIQ_COLL_RATIO_BPS`	`16000` (160%)
`CLOSE_FACTOR_BPS`	`5000` (50%)
`DEEP_TRIGGER_BPS`	`10500` (105%)
`LIQ_BONUS_BPS`	`500` (5%)
`U_MAX_BPS`	`5000`
`LENDABLE_AMM_BPS`	`3000`
VM buckets	compute 18,009 / resource 6,100 / storage 111,911
Transaction limit	16,384 bytes
gasMax range	1–99

6.3 Unit conversion

HAC (on-chain zhu):

```
HAC = zhu / 1e8
"5:249" = 5 × 10^(249−248) = 50 HAC
"1:248" = 1 HAC
```

hacds (decimal 0):

```
hacds count == the on-chain integer, no conversion
```

⚠ **Do not divide hacds by 1e8.** That was the previous edition (decimal 8).

6.4 A minimal working state-reader

```
const NODE = 'https://api.hacds.com';
const CONTRACT = 'T1A8kaeWqK6wMUw5UMHCA6tMQ1iBdECyH';

async function call(fn, params = '') {
  const q = new URLSearchParams({
    contract: CONTRACT,
    function: fn,
    params,
    caller: CONTRACT,       // omitting this fails on insufficient balance
  });
  const r = await fetch(`${NODE}/query/contract/sandboxcall?${q}`);
  const j = await r.json();
  if (j.ret !== 0) throw new Error(j.err || 'call failed');
  return j.ret_val;
}

// Pool reserves on both sides
const hacZhu = await call('amm_reserves');
const ds     = await call('amm_reserve_ds');

// Spot price (zhu per hacds)
const spot = Number(hacZhu) * 1e8 / Number(ds);
```

Concurrency note: each IP has only 2 concurrency slots. Queue bulk reads; do not use `Promise.all`.

This document corresponds to the finalized version of `HacdsDeFi3000.fitsh`, dated 2026-11-11.